นายชินะสุ สุขเจริญ 6709616459

ข้อ 1: จาก Slide 06 จงศึกษาเรื ่องการพัฒนาโปรแกรมภาษา C ตั ้งแต่หน้า 24 ถึง 29 และเขียนโปรแกรม hellomain.c hellosub1.c hellosub2.c และคอมไฟล์และ link object files เพื ่อสร้างโปรแกรม hello ในการส่งงานให้ นศ แสดง source code พร้อมทั้งผลการรันโปรแกรมในรายงาน (10 คะแนน)

1. Hellosub1.c

```
#include <stdio.h>
#include "hello1.h"

void fun1(void){
printf("hello1 %d\n", CONSTANT1);
}
```

2. Hellosub2.c

```
#include <stdio.h>
void fun2(void){
printf("hello2\n");
}
```

3.hellomain.c

```
#include <stdio.h>

extern void fun1(void);
extern void fun2(void);

int main(){
    fun1();
    fun2();
    return 0;
}
```

ผลลัพธ์ :hello1 100

hello2

ข้อ 2: ให้ นศ ค้นหาข้อมูลเกี ่ยวกับ system call (หรือกลุ ่มของ system call) ต่อไปนี ้จาก internet และอธิบายอย่างสั ้นๆว่า (1) system call เหล่านี้ทำหน้าที ่อะไร (2) ในการใช้งานต้องอ้างอิง include file อะไร (40 คะแนน)

a.fork

หน้าที่: สร้าง child process โดยการคัดลอกมาจาก parent

อ้างอิง: #include <unistd.h>

b. wait

หน้าที่: สั่งห้ parent หยุดรอจนกว่า child process ตัวใดตัวหนึ่งจะทำงานเสร็จ

อ้างอิง: #include <sys/wait.h>

c. waitpid

หน้าที่: เป็น wait ที่ระบุได้ว่าจะรอ processไหนเสร็จ(ดูPID)

อ้างอิง: #include <sys/wait.h>

d. execlp

หน้าที่: ใช้รันโปรแกรมอื่นมาทับ process ปัจจุบัน โดยค้นหาไฟล์จาก PATH และรับ list

อ้างอิง: #include <unistd.h>

e. exit

หน้าที่: ใช้จบการทำงานของ process ปัจจุบัน และ status กลับไปให้ parent

อ้างอิง: #include <stdlib.h>

f. getppid

หน้าที่: คืนค่า PID ของ parent

อ้างอิง: #include <stdlib.h>

g. getpid

หน้าที่: คืนค่าPID ของ process ปัจจุบัน

อ้างอิง: #include <unistd.h>

h. execv

หน้าที่: ใช้รันโปรแกรมอื่นมาทับ process ปัจจุบัน โดยรับอาร์กิวเมนต์ในรูปแบบของ Array (Vector)

อ้างอิง: #include <unistd.h>

ข้อ 3: จาก Slide 06 จง ค้นคว้าด้วยตนเองและตอบอย่างสั้นว่า exec*() system call มีหน้าที่ทำอะไร และ exec system calls ต่อไปนี้ มีความแตกต่างกันอย่างไร (40 คะแนน)

exec*() คือการนำโปรแกรมใหม่มาทับprocessเดิมโดยจะเคลียร์หน่วยความจำเดิมและโหลดโปรแกรมใหม่เข้าไปรันแทนที่ โดย pid ยังเหมือนเดิม โดย exec แต่ละตัวจะต่างกันตรงที่วิธีส่งค่าagrument และ วิธีการค้นหาไฟล์

l (List): ส่งargumentแบบเป็นรายการ

v (Vector): ส่งargumentแบบ Array ของ String

p (Path): ค้นหาชื่อไฟล์โปรแกรมจากระบบ PATH ของเครื่อง (ไม่ต้องระบุ Full Path)

e (Environment): อนุญาตให้กำหนด Environment variables ใหม่ให้กับโปรแกรมที่จะรัน

ข้อ 4: ให้เขียน shell script x ดัง ภาพ 1(a) และ script y ดังภาพ 1(b) ให้รัน script และกด Ctrl + C เพื่อ หยุดกรทำงานของ script แล้วตอบคำถามต่อไปนี้ (15 คะแนน)

1.จงอธิบายแต่ละคำสั่งใน script x และ y

Script x:

#!/bin/bash : ไฟล์นี้ run ด้วย bash shell

r=$(($1 - 1)) : นำ argument ตัวแรก $1 ลบ 1

echo $r : แสดงค่า r

echo PID=$$: แสดงpid ของ process ปัจจุบัน

n="./x" : กำหนดให้ n เป็นชื่อ x

$n $r : เรียกscriptตัวเอง ซ้ำ

Script y:

#!/bin/bash : ไฟล์นี้ run ด้วย bash shell

r=$(($1 - 1)) : นำ argument ตัวแรก $1 ลบ 1

echo $r : แสดงค่า r

echo PID=$$: แสดงpid ของ process ปัจจุบัน

n="./x" : กำหนดให้ n เป็นชื่อ z

exec $n $r: เรียกscriptตัวเอง ซ้ำ แต่ไม่สร้างprocess ใหม่จะเททนที่process ปัจจุบัน

2. จง capture หน้าจอผลการรัน $ ./x 200 และอธิบายว่าทำไมค่า r จึงลดลงและ เพราะเหตุใดค่า PID ในการรันแต่ละครั้งจึงเปลี่ยนไป

r ลด เพราะ คำสั่ง r=$(($1 - 1)) ให้นำ argument ไปลบ 1 เรื่อย มันเลยนำ200มา -1 และคำสั่ง $n $r ทำให้มันวนลูป ลบด้วย 1ไปเรื่อยๆ และที่ pid เปลี่ยนเพราะทุกครั้งที่เรียกตัวเองซ้ำมันจะนับเป็นprocess ใหม่ ทำให้เลขpid เปลี่ยนไปทุกครั้งที่ เรียก

3. จง capture หน้าจอผลการรัน $ ./y 200 และอธิบายว่าผลลัพธ์ของ y ต่างจากการรัน x อย่างไร เพราะเหตุใด

Y ต่างกับ ตรงที่ x exec $n $r จะไม่ได้สร้างprocess ใหม่แต่จะแทนที่ process เดิม ทำให้เลขpid เหมือนเดิม อีกทั้งยังประหยัดหน่วยความจำได้ดีกว่า x

5.1. จงอธิบายแต่ละคำสั่งใน script a และ b

ข้อ 5: ให้เขียน shell script a ดัง ภาพ 2(a) และ script b ดังภาพ 2(b) ให้รัน script และกด Ctrl + C เพื่อหยุดกรทำงานของ script แล้วตอบคำถามต่อไปนี้ (25 คะแนน)

Script a:

#!/bin/bash : ไฟล์นี้ run ด้วย bash shell

m=$((${m} - 1)) : นำ ค่า m มาลบ 1 แล้วเก็บกลับที่เดิม m = m - 1

echo $m : แสดงค่า m

echo PID=$$: แสดงpid ของ process ปัจจุบัน

${0} :รันตัวเองซ้ำ โดยสร้าง process ใหม่เรื่อยๆ

Script b:

#!/bin/bash : ไฟล์นี้ run ด้วย bash shell

m=$((${m} - 1)) : นำ ค่า m มาลบ 1 แล้วเก็บกลับที่เดิม m = m - 1

echo $m : แสดงค่า m

echo PID=$$: แสดงpid ของ process ปัจจุบัน

${0} :รัยตัวเองซ้ำโดยแทนที่ process เดิม

5.2. จงตอบว่าจะต้องรันคำสั่งอะไร ก่อนรันคำสั่ง $ ./a จึงจะทำให้ผลลัพธ์ (ไม่รวม ค่า PID ที ่ไม่ต้องเหมือนกัน แต่ค่า PID สามารถเพิ ่มขึ ้นได้เหมือนของ ./x) เหมือนรัน และเพราะดหตุใดจึงเป็นเช่นนั้น $ ./x 200

Ans.คำสั่งexport m=200 เพราะ script a เรียกใช้ตัวแปรชื่อ m โดยตรง ไม่ได้เรียกผ่าน parameter ลำดับ1 เหมือน script x

5.3. จง capture หน้าจอผลการรัน $ ./a และอธิบายว่าทำไมค่า m จึงลดลงและ เพราะเหตุใดค่า PID ใน การรัน แต่ละครั้งจึงเปลี่ยนไป

m ลด เพราะ คำสั่ง m=$((${m} - 1)) ให้นำ m ไปลบ 1 เรื่อย มันเลยนำ200(ที่export ไว้)มา -1 และคำสั่ง ${0} ทำให้มันวนลูป ลบด้วย 1ไปเรื่อยๆ และที่ pid เปลี่ยนเพราะทุกครั้งที่เรียกตัวเองซ้ำมันจะนับเป็นprocess ใหม่ ทำให้เลขpid เปลี่ยนไปทุกครั้งที่ เรียก

5.4. จง capture หน้าจอผลการรัน $ ./b อธิบายว่าผลลัพธ์ต่างจากการรัน ./a อย่างไร เพราะเหตุใด

b ต่างกับ ตรงที่ a exec ${0} จะไม่ได้สร้างprocess ใหม่แต่จะแทนที่ process เดิม ทำให้เลขpid เหมือนเดิม อีกทั้งยังประหยัดหน่วยความจำได้ดีกว่า a

5.5. ถ้าต้องการให้การประมวลผลของ ./b หยุดที่ค่า m = -200 และพิมพ์คำว่า “done” ออกสู่ หน้าจอจะต้องเปลี่ยน script b อย่างไร อธิบายคำสั่งที่ใช้และ capture ผลการรัน

เพิ่มคำสั่งดังนี้

```
if [ $m -eq -200 ]; then
   echo "done"
else
   exec ${0}
fi
```

ข้อ 6: จาก Slide 06 จงเขียนและรันตัวอย่างโปรแกรมต่อไปนี้ และอธิบายสั้นๆว่าโปรแกรมเขียนขึ้นเพื่อแสดง การประมวลผลอะไร (140 คะแนน)

ex1.c : เขียนมาเพื่อ ดูการทำงานของ System Call fork() ซึ่งทำหน้าที่ในการclone ตัวเองมาช่วยprocess ต่อ

ex2.c : เหมือนex1.c แต่เพิ่ม fork() ไปอีกตัว มันเลยกลายเป็น การแตกตัวของprocessแบบทวีคูณ จาก 1->2 และ 2-> 4

ex2-gen.c : คำนวณจำนวนprocessเกิดขึ้นจากการใช้ fork() ภายในลูป for

ex3.c : คุมลำดับการทำงานระหว่างprocess parent และ child โดยใช้คำสั่งสำคัญคือ waitpid()

ex4.c : ใช้ sleep() ที่มีระยะเวลาต่างกันมากระหว่างparent และ child ทำให้เกิด **orphan process**

ps -fu :parent หายไปแล้วแต่ยังเห็น child อยู่ แต่ PPID จะเปลี่ยนไป

ps -ef :parent ตายแล้วchild จะถูกส่งไปให้ init และ PPID ของ child จะเปลี่ยน เป็น1

ex5.c :

ps -fu : จะเห็นรายละเอียดprocess ในคอลัมน์ **CMD** หรือชื่อ child process จะมีคำว่า **<defunct>** ปรากฏต่อท้าย ในคอลัมน์ **S** หรือ **STAT** ของchild จะขึ้นเป็นตัว **Z**

ps -ef : child ยังมี **PID** ค้างอยู่ และ คอลัมน์ **PPID** ของลูกจะยังคงเป็น **เลข PID ของparent**

ex5-1.c และ ex5-2.c :ex5-1 มีสถานะ R ส่วน ex5-2 มีสถานะ S สาเหตุที่ต่างกันเพราะparent ใช้คำสั่งต่างกัน ตัวแรกใช้ for ทำให้กลายเป็น Zombie ตลอดไปเพราะ parent จะประมวลผลไปเรื่อยๆ ส่วนตัวที่2 จะใช้ sleep ทำให้หลับยาวแทน ทำให้ประหยัด cpu ไปเยอะ

ex6.c : โปรแกรมปรพะมวลผลใช้ waitpid เพื่อทำ Synchronization เพื่อให้มั่นใจว่าparent จะจบหลัง child เสมอ

ex7.c : โปรแกรมประมวลผลการทำ multiprocessing ที่parent ทำหน้าที่เป็นผู้ควบคุมสร้างงานให้child หลายตัวทำหลายงานพร้อมกัน และรอจนกว่างานทุกอย่างจะเสร็จก่อนจะจบโปรแกรม

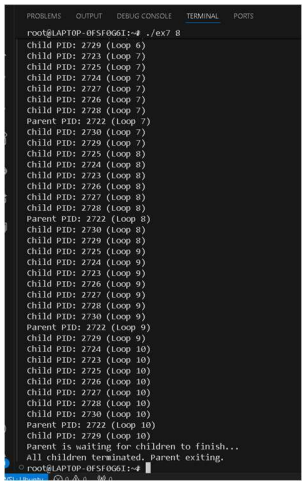
ex8.c : เขียนมาเพื่อ ดูการทำงานของ System Call exec() ซึ่งทำหน้าที่ในการแทนที่process(PID เดิม)

ex9.c : ประมวลผลการทำ Process Overlay แบบวนซ้ำ"โดยใช้ System Call execlp

ex10.c: โปรแกรมแสดงกระบวนการ สร้างprocessใหม่เพื่อรันโปรแกรมอื่น โดยที่parentยังอยู่ โดดยใช้ fork() ร่วมกับ exec()

shell1.c : โปรแกรมจำลองการทำงานของหน้าจอ Command Line (Terminal) แบบพื้นฐาน

ข้อ 7: จากโปรแกรม ex7.c ให้ขยายโปรแกรมให้สร้าง โพรเซสลูกจำนวน n โพรเซส โดยรับค่า n เป็นค่า Parameter ตอนเรียกโปรแกรม สมมุติว่าคแมไพล์โปรแกรมเป็น ex7 executable เมื่อเรียก $ ./ex7 8 หมายถึงค่า n = 8 โดยที่โพรเซสลูกทั้งหมดพร้อมทั้ง parent โพรเซสจะต้อง พิมพ์ค่า Process ID ของตนเอง ออกสู่หน้าจอเป็นจำนวน 10 ครั้ง ไปพร้อมๆกัน แล้วหลังจากนั้นให้ Parent Process ใช้ waitpid system call รอให้ลูกทั้งหมดจบการทำงาน ดังภาพที่ 3 ให้แสดง source code และ capture ผลลัพธ์ใส่ใน รายงาน (40 คะแนน)



Source code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>


int main(int argc, char *argv[]) {

  if (argc < 2) {

    printf("Usage: %s <number_of_processes>\n", argv[0]);

    return 1;

  }


  int n = atoi(argv[1]);

  pid_t pids[n];
```

```
for (int i = 0; i < n; i++) {
   pids[i] = fork();

   if (pids[i] < 0) {
      perror("Fork failed");
      exit(1);
   }
   else if (pids[i] == 0) {
      // --- ส่วนของ Child Process ---
      for (int j = 0; j < 10; j++) {
         printf("Child PID: %d (Loop %d)\n", getpid(), j + 1);
         usleep(100000);
      }
      exit(0);
   }
}

for (int j = 0; j < 10; j++) {
   printf("Parent PID: %d (Loop %d)\n", getpid(), j + 1);
   usleep(100000);
}
printf("Parent is waiting for children to finish...\n");
for (int i = 0; i < n; i++) {
   int status;
   waitpid(pids[i], &status, 0);
}
```

```
    printf("All children terminated. Parent exiting.\n");
    return 0;
}
```

ข้อ 8: จงเขียนโปรแกรม ให้สร้างโปรเซสลูกซ้อนกันดังภาพที่ 4 จำนวน n ชั้น (โดยรับค่า n จาก Command Line Parameter เหมือนในข้อก่อนหน้า) นศ สามารถใช้วิธีการ recursive หรือวิธีการ loop ก็ได้ เลือกวิธีใด วิธีหนึ่ง (หลังจากเลือกแล้ว นศ อาจลองพิจารณาด้วยว่าถ้าจะทำวิธีที่ไม่ได้เลือกจะต้องเขียนโปรแกรมอย่างไร) มีข้อกำหนดดังนี้ ให้แสดง source code และ capture ผลลัพธ์ใส่ในรายงาน (40 คะแนน)

- สร้างโปรเซสลูก เรียกว่าโปรเซส Gen 1 แล้วให้ main รอให้ลูกจบการประมวลผล แล้ว
- ให้โปรเซสลูก fork สร้างโปรเซสหลาน เรียกว่าโปรเซส Gen 2 แล้วให้โปรเซส Gen 1 รอให้ Gen2 จบ การประมวลผล และ
- ให้โปรเซส Gen 2 fork โปรเซส Gen 3 แล้วให้โปรเซส Gen 2 รอให้ Gen 3 จบการประมวลผล และ
- ให้โปรเซส Gen 3 fork โปรเซส Gen 4 แล้วให้โปรเซส Gen 3 รอให้ Gen 4 จบการประมวลผล และ
- ทำเช่นนี้ไปเรื่อยๆ ให้โปรเซส Gen N fork โปรเซส Gen N+1 แล้วให้โปรเซส Gen N รอให้ Gen N+1 จบการประมวลผล และ

• ให้แต่ละโปรเซสพิมพ์ค่า pid ของตนเอง (อาจใช้ getpid()) “My PID is \n” และค่า pid ของโปรเซส parent (อาจใช้ getppid()) “My PID is \n” ให้ นศ พิมพ์โค๊ดของโปรแกรมใส่ในรายงานพร้อมทั้งเขียนคอมเม้นท์อธิบายคร่าวๆ และให้ capture หน้าจอผลการประมวลผลและอธิบายจากผลการรันว่าทำไมโปรแกรมถึงทำงานได้ถูกต้องตามที่ต้องการ

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
  // เช็คว่ามีการรับค่า  n จากkeyboardมั้ย
  if (argc < 2) {
    printf("Usage: %s <number_of_generations>\n", argv[0]);
    return 1;
  }

  int n = atoi(argv[1]); // แปลง string เป็น int
  int current_gen = 0;

  for (int i = 1; i <= n; i++) {
    pid_t pid = fork();

    if (pid < 0) {
```

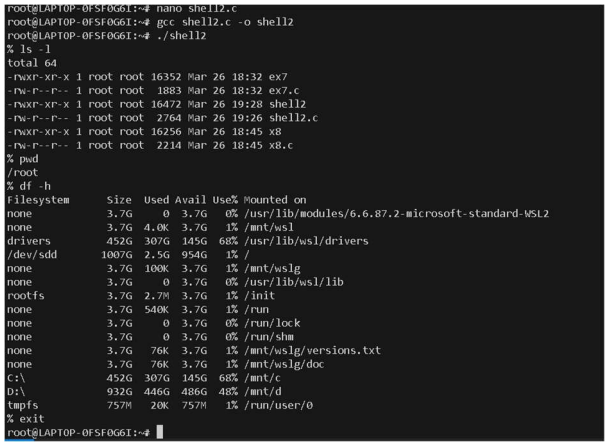
```
    // กรณี fork ผิดพลาด
    perror("fork failed");
    exit(1);
}
else if (pid == 0) {
    // Child Process
    current_gen = i;
    // วนลูปต่อเพื่อไปสร้างchildของตัวเอง (Gen ถัดไป)
    continue;
}
else {
    //  Parent Process
    // รอ child ที่มีpid ตามที่กำหนดทำงานเสร็จก่อน
    waitpid(pid, NULL, 0);

    // child เสร็จแล้ว parant จึงจะพิมพ์ค่าของตัวเอง
    if (current_gen == 0) {
        printf("Main Process: My PID is %d\n", getpid());
    } else {
        printf("Gen %d: My PID is %d, Parent PID is %d\n", current_gen, getpid(), getppid());
    }

    // พิมพ์เสร็จแล้วให้จบprocessของ parentทันที เพื่อไม่ให้หลุดไปวนลูป fork ต่อ
```

```
        exit(0);
      }
    }

    // สำหรับprocess gen สุดท้ายที่ไม่มีการforkต่อแล้ว
    if (current_gen == n) {
      printf("Gen %d (Last): My PID is %d, Parent PID is %d\n", current_gen, getpid(), getppid());
    }

    return 0;
}
```

```
root@LAPTOP-0FSF0G6I:~# ./x8 3
Gen 3 (Last): My PID is 7449, Parent PID is 7448
Gen 2: My PID is 7448, Parent PID is 7447
Gen 1: My PID is 7447, Parent PID is 7446
Main Process: My PID is 7446
```

ผลลัพธ์ที่ได้จะแสดงผลจาก **Gen n** ย้อนกลับมาหา Main ซึ่งตรงตาม Flow ที่ระบุว่าchildต้องพิมพ์และจบการทำงานก่อนที่parent ที่รอจากคำสั่ง waitpid จึงจะทำงานต่อได้

้อ 9: จากโปรแกรม shell1.c ใน Slide ขอให้ขยายโปรแกรมเป็น shell2.c โดยมีข้อกำหนดดังนี้ (40 คะแนน) 1. ห้ามใช้ stdlib function system() 2. โปรแกรม shell2.c ต้องสามารถรับค่า parameters ได้ (Hint: ใช้ strtok() หรือวิธีการตัดคำอย่าง อื่น) ตัวอย่างคำสั่งเช่น $ ls -l $ df -h เป็นต้น 3. ถ้าผู้ใช้ป้อน string “exit” ให้ shell2 จบการประมวลผล 4. ถ้าผู้ใช้ป้อน string “pwd” ให้แสดงค่า current directory (Hint: ใช้ getcwd() system call)



```
root@LAPTOP-0FSF0G6I:~# nano shell2.c
root@LAPTOP-0FSF0G6I:~# gcc shell2.c -o shell2
root@LAPTOP-0FSF0G6I:~# ./shell2
% ls -l
total 64
-rwxr-xr-x 1 root root 16352 Mar 26 18:32 ex7
-rw-r--r-- 1 root root  1883 Mar 26 18:32 ex7.c
-rwxr-xr-x 1 root root 16472 Mar 26 19:28 shell2
-rw-r--r-- 1 root root  2764 Mar 26 19:26 shell2.c
-rwxr-xr-x 1 root root 16256 Mar 26 18:45 x8
-rw-r--r-- 1 root root  2214 Mar 26 18:45 x8.c
% pwd
/root
% df -h
Filesystem      Size  Used Avail Use% Mounted on
none            3.7G     0  3.7G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.7G  4.0K  3.7G   1% /mnt/wsl
drivers         452G  307G  145G  68% /usr/lib/wsl/drivers
/dev/sdd       1007G  2.5G  954G   1% /
none            3.7G  100K  3.7G   1% /mnt/wslg
none            3.7G     0  3.7G   0% /usr/lib/wsl/lib
rootfs          3.7G  2.7M  3.7G   1% /init
none            3.7G  540K  3.7G   1% /run
none            3.7G     0  3.7G   0% /run/lock
none            3.7G     0  3.7G   0% /run/shm
none            3.7G   76K  3.7G   1% /mnt/wslg/versions.txt
none            3.7G   76K  3.7G   1% /mnt/wslg/doc
C:\             452G  307G  145G  68% /mnt/c
D:\             932G  446G  486G  48% /mnt/d
tmpfs           757M   20K  757M   1% /run/user/0
% exit
root@LAPTOP-0FSF0G6I:~#
```

Source code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <sys/wait.h>



#define MAXLINE 500

#define MAXARGS 50
```

```
int main(void) {
    char buf[MAXLINE];
    pid_t pid;
    int status;
    char *args[MAXARGS];
    char cwd[1024];

    // แสดง Prompt เริ่มต้น
    printf("%% ");

    // วนลูปรับคำสั่งจาก User ผ่าน stdin
    while (fgets(buf, MAXLINE, stdin) != NULL) {

        // ตัดตัวขึ้นบรรทัดใหม่ (\n) ที่ติดมากับ fgets ออก
        if (buf[strlen(buf) - 1] == '\n')
            buf[strlen(buf) - 1] = 0;

        // แยกคำสั่งและ Parameter ด้วย strtok
        int i = 0;
        args[i] = strtok(buf, " ");
        while (args[i] != NULL && i < MAXARGS - 1) {
            i++;
            args[i] = strtok(NULL, " ");
        }
```

```
// ตัวสุดท้ายของarray args ต้องเป็น NULL เสมอสำหรับ execvp
args[i] = NULL;


// กรณีผู้ใช้กด Enter เปล่าๆ ให้ข้ามไปแสดง prompt ใหม่
if (args[0] == NULL) {
    printf("%% ");
    continue;
}
//  userพิมพ์ "exit" ให้จบการทำงาน
if (strcmp(args[0], "exit") == 0) {
    break;
}


//userพิมพ์ pwd ให้แสดง Path ปัจจุบัน
if (strcmp(args[0], "pwd") == 0) {
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("getcwd error");
    }
    printf("%% ");
    continue; // runเสร็จแล้วข้ามไปรับคำสั่งถัดไป
}
```

```
    // สร้าง child process เพื่อรันคำสั่งพร้อม Parameter
    if ((pid = fork()) < 0) {
        printf("fork error\n");
        exit(1);
    } else if (pid == 0) {
        /* Child)*/
        // ใช้ execvp เพื่อรองรับการรับarrayสั่ง
        execvp(args[0], args);


        // กรณี execvp ทำงานพลาด
        printf("couldn't execute: %s\n", args[0]);
        exit(127);
    }


    /* Parent)*/
    // รอให้childทำงานให้เสร็จก่อน
    if ((pid = waitpid(pid, &status, 0)) < 0)
        printf("waitpid error\n");


    printf("%% "); // พิมพ์ prompt รอรับคำสั่งใหม่
}


printf("Shell exited.\n");
exit(0);
```

}